

# DOCUMENTO DE ARQUITECTURA

## Search Microservice

Sprint 1

---

Versión 1.0 | Marzo 2026

Equipo de Desarrollo

# 1. Introducción

---

## 1.1 Propósito del Documento

Este documento describe la arquitectura técnica del Search Microservice, un componente especializado dentro de una plataforma de e-commerce basada en microservicios. Su objetivo es servir como referencia de diseño para el equipo de desarrollo durante el desarrollo del producto software y como insumo para el Scrum Master en la elaboración del informe final del sprint.

## 1.2 Alcance

El microservicio cubre las siguientes capacidades:

- Búsqueda full-text de productos por nombre, descripción y marca.
- Filtros dinámicos por precio, categoría, disponibilidad y rating.
- Autocompletado inteligente.
- Sugerencias relacionadas.
- Ranking por relevancia con tolerancia a errores tipográficos (fuzzy search).
- Indexación reactiva basada en eventos de dominio publicados por el Catalog Microservice.

## 1.3 Contexto del Sistema

El Search Microservice opera dentro de una arquitectura de microservicios y aplica el patrón CQRS (Command Query Responsibility Segregation), actuando exclusivamente como Read Model optimizado. No modifica datos ni consulta directamente la base de datos del Catalog Service.

## 2. Requerimientos

### 2.1 Requerimientos Funcionales

| ID    | Nombre                   | Descripción                                                                                                | Prioridad |
|-------|--------------------------|------------------------------------------------------------------------------------------------------------|-----------|
| RF-01 | Búsqueda full-text       | El sistema debe permitir buscar productos por nombre, descripción y marca utilizando Elasticsearch.        | Alta      |
| RF-02 | Filtros dinámicos        | El sistema debe soportar filtros por precio (rango), categoría, disponibilidad y rating.                   | Alta      |
| RF-03 | Ordenamiento             | El sistema debe ordenar resultados por precio ascendente/descendente y por popularidad.                    | Media     |
| RF-04 | Autocompletado           | El sistema debe sugerir términos de búsqueda mientras el usuario escribe (mínimo 3 caracteres).            | Alta      |
| RF-05 | Sugerencias relacionadas | El sistema debe devolver productos relacionados a la búsqueda actual.                                      | Media     |
| RF-06 | Fuzzy search             | El sistema debe tolerar errores tipográficos en los términos de búsqueda.                                  | Media     |
| RF-07 | Ranking por relevancia   | Los resultados deben ordenarse por relevancia calculada por Elasticsearch.                                 | Alta      |
| RF-08 | Indexación por eventos   | El microservicio debe consumir eventos ProductCreated y ProductUpdated de RabbitMQ para indexar productos. | Alta      |
| RF-09 | Paginación de resultados | La API debe soportar paginación con parámetros page y pageSize.                                            | Alta      |

### 2.2 Requerimientos No Funcionales

| ID     | Nombre                | Descripción                                                                                                             | Prioridad |
|--------|-----------------------|-------------------------------------------------------------------------------------------------------------------------|-----------|
| RNF-01 | Rendimiento           | El tiempo de respuesta del 95% de las consultas (P95) debe ser menor a 100 ms.                                          | Alta      |
| RNF-02 | Disponibilidad        | El servicio debe tener alta disponibilidad mediante réplicas de Elasticsearch.                                          | Alta      |
| RNF-03 | Escalabilidad         | La arquitectura debe soportar escalabilidad horizontal (múltiples instancias).                                          | Alta      |
| RNF-04 | Consistencia eventual | Se acepta consistencia eventual; no se requiere consistencia fuerte entre Catalog y Search.                             | Media     |
| RNF-05 | Seguridad             | Todas las peticiones deben validar JWT a través de Kong/Keycloak. Se aplica rate limiting y sanitización de parámetros. | Alta      |
| RNF-06 | Observabilidad        | Las métricas clave (P95, cache hit rate, QPS, latencia ES) deben estar disponibles en Prometheus/Grafana.               | Media     |
| RNF-07 | Límite de paginación  | Se debe evitar deep pagination usando el mecanismo search_after de Elasticsearch.                                       | Baja      |

## 3. Backlog e Historias de Usuario

### 3.1 Relación Requerimientos – Historias de Usuario

Cada historia de usuario (HU) está trazada directamente a los requerimientos funcionales definidos en la sección anterior. Los puntos de historia siguen la escala Fibonacci.

| HU    | Nombre                   | Historia                                                                                                                | RF Relacionados     | Prioridad |
|-------|--------------------------|-------------------------------------------------------------------------------------------------------------------------|---------------------|-----------|
| HU-01 | Búsqueda por keyword     | Como cliente, quiero buscar productos por palabra clave para encontrar rápidamente lo que necesito.                     | RF-01, RF-06, RF-07 | Alta      |
| HU-02 | Filtros de búsqueda      | Como cliente, quiero filtrar por precio y categoría para ajustar los resultados a mi presupuesto.                       | RF-02, RF-03        | Alta      |
| HU-03 | Autocompletado           | Como cliente, quiero que el sistema sugiera productos mientras escribo para ahorrar tiempo.                             | RF-04, RF-05        | Alta      |
| HU-04 | Paginación de resultados | Como cliente, quiero navegar entre páginas de resultados para explorar todas las opciones disponibles.                  | RF-09               | Media     |
| HU-05 | Indexación automática    | Como administrador, quiero que los productos nuevos y actualizados aparezcan en los resultados sin intervención manual. | RF-08               | Alta      |

### 3.2 Criterios de Aceptación por Historia

#### HU-01 · Búsqueda por keyword

- Dado un término de búsqueda, el sistema retorna productos relevantes en menos de 100 ms.
- Los resultados con errores tipográficos de hasta 2 caracteres siguen apareciendo (fuzzy).
- El endpoint GET /api/search?q={término} responde con estructura paginada.

#### HU-02 · Filtros de búsqueda

- Los filtros de precio, categoría, disponibilidad y rating pueden combinarse entre sí.
- El ordenamiento por precio ascendente/descendente funciona correctamente.
- Los parámetros inválidos retornan error 400 con mensaje descriptivo.

#### HU-03 · Autocompletado

- Con mínimo 3 caracteres escritos, el endpoint GET /api/search/suggest?q={texto} retorna al menos 3 sugerencias.
- Las sugerencias se entregan en menos de 50 ms gracias al caché de Redis.

HU-05 · Indexación automática

- Cuando el Catalog Service publica un evento ProductCreated, el producto aparece en búsquedas en un máximo de 5 segundos.
- Cuando se publica ProductUpdated, los datos actualizados se reflejan y el caché de Redis asociado se invalida.

4. Arquitectura del Sistema

4.1 Patrón Arquitectónico: CQRS + Clean Architecture

El Search Microservice implementa el patrón CQRS, separando responsabilidades de escritura (indexación) y lectura (consulta). Actúa exclusivamente como Read Model; el Write Model reside en el Catalog Service.

Internamente, el microservicio sigue los principios de Clean Architecture, organizando el código en capas concéntricas con dependencias hacia adentro:

- Domain: entidades de búsqueda y contratos de repositorio.
- Application: casos de uso (SearchProducts, SuggestProducts, IndexProduct).
- Infrastructure: implementaciones de Elasticsearch, Redis y el consumer de RabbitMQ.
- Presentation: controladores REST (Kong → API).

4.2 Bases de Datos y Su Rol

| Tecnología    | Tipo              | Rol                                                                                                   | Casos de uso        |
|---------------|-------------------|-------------------------------------------------------------------------------------------------------|---------------------|
| Elasticsearch | Motor de búsqueda | Almacena y consulta el índice de productos. Provee full-text search, fuzzy search, filtros y ranking. | HU-01, HU-02, HU-03 |
| Redis         | Caché en memoria  | Cachea queries frecuentes, resultados paginados y filtros populares para cumplir SLA < 100 ms.        | HU-01, HU-02, HU-03 |

4.3 Flujo de Indexación (Write Path)

La indexación es completamente reactiva y orientada a eventos. Search nunca consulta MongoDB directamente.

- 1. El Catalog Service crea o actualiza un producto en MongoDB.
- 2. Publica un evento ProductCreated o ProductUpdated en RabbitMQ.
- 3. El Search Service consume el evento desde la cola.
- 4. Transforma el modelo de catálogo al modelo de búsqueda (SearchDocument).
- 5. Indexa el documento transformado en Elasticsearch.
- 6. Invalida las entradas de caché afectadas en Redis (TTL o eliminación directa).

4.4 Flujo de Consulta (Read Path)

- 1. El cliente realiza una petición HTTP al API Gateway (Kong).

- 2. Kong valida el JWT con Keycloak antes de enrutar la petición.
- 3. El request llega al Search Microservice.
- 4. Search consulta Redis buscando un resultado cacheado para esa query.
- 5a. Si existe en caché: retorna la respuesta directamente.
- 5b. Si no existe: consulta Elasticsearch, guarda el resultado en Redis con TTL y retorna la respuesta.

## 4.5 Modelo del Índice en Elasticsearch

El documento indexado tiene la siguiente estructura y tipado:

| Campo       | Tipo ES         | Descripción                                                                            | Índice recomendado |
|-------------|-----------------|----------------------------------------------------------------------------------------|--------------------|
| productId   | keyword         | Identificador único del producto, sin análisis de texto.                               | No                 |
| name        | text + analyzer | Nombre del producto. Permite búsqueda full-text y fuzzy. Incluye analizador de idioma. | Si                 |
| description | text            | Descripción detallada. Indexada para búsqueda full-text.                               | No                 |
| category    | keyword         | Categoría del producto. Usado para filtros exactos.                                    | Si                 |
| price       | float           | Precio del producto. Usado para filtros de rango y ordenamiento.                       | Si                 |
| rating      | float           | Calificación promedio. Usado para filtros y ordenamiento por popularidad.              | Si                 |
| available   | boolean         | Indica si el producto está disponible. Filtro booleano.                                | No                 |
| brand       | keyword         | Marca del producto. Filtro exacto y búsqueda por marca.                                | Si                 |

## 4.6 Diseño de API REST

### Búsqueda Principal

GET

/api/search?q={término}&category={cat}&minPrice={n}&maxPrice={n}&page={n}&pageSize={n}&sort={precio\_asc|precio\_desc|popularidad}

### Autocompletado

GET /api/search/suggest?q={texto\_parcial}

### Ejemplo autocompletado:

solicitud: GET /api/search/suggest?q={Zap}

Devuelve: Zapatos deportivos Zapatos casuales Zapatos Nike

Respuesta estándar de búsqueda:

- `totalResults`: número total de resultados encontrados.
- `page`: página actual.
- `pageSize`: cantidad de resultados por página.
- `products`: arreglo de productos con sus atributos indexados.

### 4.7 Canales de Comunicación

| Canal              | Tecnología       | Uso                                                                                                             |
|--------------------|------------------|-----------------------------------------------------------------------------------------------------------------|
| Entrada de eventos | RabbitMQ         | Recibe eventos <code>ProductCreated</code> y <code>ProductUpdated</code> desde <code>Catalog Service</code> .   |
| Entrada HTTP       | Kong API Gateway | Recibe peticiones REST de clientes externos, previo paso por validación JWT con Keycloak.                       |
| Autenticación      | Keycloak (JWT)   | Kong valida tokens JWT antes de enrutar. El <code>Search Service</code> no gestiona autenticación directamente. |

### 4.8 Seguridad

- Validación de JWT delegada a Kong/Keycloak en el API Gateway.
- Rate limiting configurado en Kong para prevenir abuso del servicio.
- Sanitización de todos los parámetros de búsqueda para prevenir inyecciones.
- Límites de tamaño máximo en el parámetro `q` (longitud de query).
- Protección contra queries maliciosas (query injection en DSL de Elasticsearch).

### 4.9 Observabilidad

En producción el servicio se monitorea con Prometheus y Grafana. Las métricas clave son:

- Tiempo promedio de búsqueda P95 — objetivo: < 100 ms.
- Tasa de cache hit en Redis (%) — objetivo: > 70%.
- Queries por segundo (QPS).
- Tiempo de indexación desde evento hasta disponibilidad en ES.
- Tasa de errores de búsqueda.
- Tamaño del índice de Elasticsearch.
- Latencia de Elasticsearch por tipo de query.

## 5. Administración de Git

### 5.1 Estrategia de Ramas

- main: rama principal estable, solo recibe merges desde develop con PR aprobado.
- develop: rama de integración del sprint. Base para todas las feature branches.
- feature/{hu-id}-{descripción}: una rama por historia de usuario. Ejemplo: feature/hu-01-busqueda-keyword.
- fix/{descripción}: ramas para corrección de bugs detectados en el sprint.

### 5.2 Convención de Commits

Se sigue la especificación Conventional Commits:

- feat(HU-01): implementar búsqueda full-text con Elasticsearch
- fix(HU-03): corregir timeout en suggest al consultar Redis
- chore: actualizar configuración de Docker Compose
- test(HU-02): agregar pruebas unitarias para filtros de precio

### 5.3 Code Review

- Todo PR requiere mínimo 1 aprobación de otro miembro del equipo.
- El Scrum Master revisa que el PR referencia la HU correspondiente.
- No se permiten merges directos a develop sin PR.

## 6. Matriz de Riesgos

### 6.1 Criterios de Evaluación

Probabilidad: Alta / Media / Baja. Impacto: Alto / Medio / Bajo. Nivel = combinación de ambos factores.

| ID    | Prob. | Impacto | Nivel   | Descripción                                                                                                                              | Mitigación                                                                                                                               |
|-------|-------|---------|---------|------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------|
| RF-01 | Alta  | Alta    | Crítico | Curva de aprendizaje en Elasticsearch. El equipo puede no tener experiencia suficiente con el motor de búsqueda.                         | Asignar sesión de capacitación en la primera semana. Designar un miembro referente de ES. Usar documentación oficial y ejemplos del PDF. |
| RF-02 | Media | Alta    | Alto    | Latencia de sincronización de eventos RabbitMQ. Los eventos pueden llegar con retraso o perderse, causando inconsistencias en el índice. | Implementar retry automático en el consumer. Configurar dead-letter queue. Monitorear lag de cola.                                       |
| RF-03 | Media | Media   | Medio   | Configuración de Kong/Keycloak. La integración del API Gateway puede generar bloqueos en el desarrollo local.                            | Configurar entorno local con Docker Compose que incluya Kong y Keycloak. Documentar variables de entorno requeridas.                     |
| RF-04 | Baja  | Alta    | Medio   | Rendimiento insuficiente de Redis. El caché puede no ser suficiente                                                                      | Definir TTL adecuado desde el inicio. Realizar pruebas de carga al                                                                       |



| ID    | Prob. | Impacto | Nivel | Descripción                                                                                                                                      | Mitigación                                                                                                                                   |
|-------|-------|---------|-------|--------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------|
|       |       |         |       | para alcanzar el SLA de 100 ms bajo carga alta.                                                                                                  | finalizar el sprint. Ajustar estrategia de caché si se detectan cuellos de botella.                                                          |
| RF-05 | Media | Media   | Medio | Cambios en el esquema del evento de dominio. El Catalog Service puede modificar la estructura de ProductCreated/ProductUpdated sin previo aviso. | Definir contrato de evento (versión 1.0) con el equipo de Catalog. Usar validación de esquema en el consumer.                                |
| RF-06 | Baja  | Media   | Bajo  | Deep pagination degradando el rendimiento de Elasticsearch ante solicitudes de páginas muy lejanas.                                              | Implementar search_after de Elasticsearch en lugar de offset desde el inicio del sprint. No habilitar paginación clásica para páginas > 100. |
| RF-07 | Alta  | Baja    | Medio | Ausencia de miembros del equipo durante el sprint de 15 días.                                                                                    | Documentar tareas diariamente. Mantener el tablero JIRA actualizado. El Scrum Master redistribuye carga si hay ausentismo.                   |

6.2 Nivel de Riesgo General del Sprint

Basado en la matriz, el nivel de riesgo general del Sprint 1 se clasifica como MEDIO-ALTO. Los riesgos críticos y altos (RF-01, RF-02) deben atenderse en la primera semana del sprint para no comprometer la entrega.

7. Estrategia de Performance

7.1 Estrategias Clave

- Redis como caché de queries frecuentes, resultados paginados y filtros populares.
- Índices optimizados en Elasticsearch con los tipos de campo correctos (keyword vs text).
- Sharding en Elasticsearch para distribuir la carga de indexación y consulta.
- Réplicas de Elasticsearch para garantizar alta disponibilidad.
- Uso de search\_after en lugar de from/size para evitar deep pagination.

7.2 Métricas clave y SLA Definidos

| Métrica                   | Objetivo                   | Medición             |
|---------------------------|----------------------------|----------------------|
| Latencia P95 búsqueda     | Por definir (< 100 ms)     | Prometheus / Grafana |
| Latencia autocompletado   | Por definir(< 50 ms)       | Prometheus / Grafana |
| Cache hit rate Redis      | Por definir (> 70%)        | Prometheus / Grafana |
| Tiempo de indexación      | Por definir (< 5 segundos) | Logs de consumer     |
| QPS (Queries por segundo) | Por definir                | Prometheus / Grafana |
| Errores de búsqueda       | Por definir                | Prometheus / Grafana |

| Métrica                | Objetivo    | Medición             |
|------------------------|-------------|----------------------|
| Tamaño del índice      | Por definir | Prometheus / Grafana |
| Latencia Elasticsearch | Por definir | Prometheus / Grafana |

## 8. Glosario

| Término       | Definición                                                                                                                   |
|---------------|------------------------------------------------------------------------------------------------------------------------------|
| CQRS          | Command Query Responsibility Segregation. Patrón que separa operaciones de escritura (commands) de las de lectura (queries). |
| Read Model    | Modelo de datos optimizado exclusivamente para consultas. No tiene responsabilidad de escritura.                             |
| Elasticsearch | Motor de búsqueda distribuido basado en Lucene. Provee búsqueda full-text, filtros, y ranking por relevancia.                |
| Redis         | Base de datos en memoria de tipo clave-valor. Usada como caché para reducir la latencia.                                     |
| RabbitMQ      | Message Broker que implementa el protocolo AMQP. Permite comunicación asíncrona entre microservicios mediante eventos.       |
| Fuzzy Search  | Búsqueda tolerante a errores tipográficos. Retorna resultados aunque el término tenga diferencias menores al esperado.       |
| JWT           | JSON Web Token. Estándar para transmitir información de autenticación de forma segura entre partes.                          |
| Kong          | API Gateway que gestiona enrutamiento, autenticación, rate limiting y otras políticas de acceso.                             |
| Keycloak      | Servidor de identidad y acceso (IAM) que gestiona autenticación y autorización mediante estándares OAuth2/OIDC.              |
| TTL           | Time To Live. Tiempo de expiración de una entrada en caché de Redis.                                                         |
| search_after  | Mecanismo de paginación de Elasticsearch que evita degradación de rendimiento en páginas profundas.                          |
| P95           | Percentil 95. Indica el tiempo máximo del 95% de las peticiones; el 5% restante puede ser más lento.                         |